

Part 2

lesson

6

受動ブザー

概要

このレッスンでは、受動ブザーを使用する方法を学習します。

実験の目的は、アルト・ド (523Hz)、Re (587Hz)、Mi (659Hz)、Fa (698Hz)、So (784Hz)、La (880Hz)、Si (988Hz) から Treble Do (1047Hz)。

必要な構成部品:

- (1) x Elegoo Uno R3
- (1) x Passive buzzer
- (2) x F-M wires (Female to Male DuPont wires)



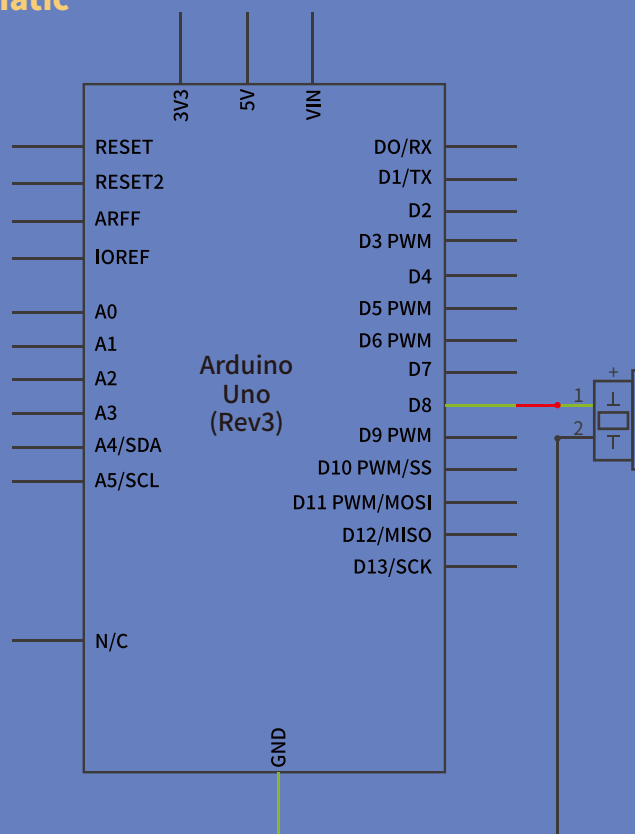
部品の紹介

Passive Buzzer:

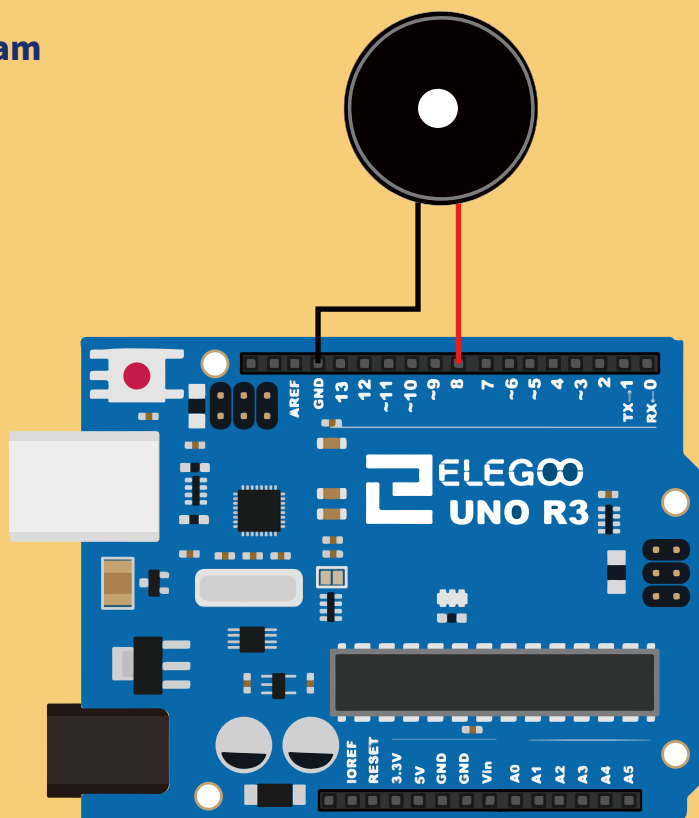
- 受動ブザーの作動原理は、空気を振動させるために PWM 生成オーディオを使用することである。振動周波数がある限り適切に変更され、異なる音を発生させることができます。たとえば、523Hz のパルスを送信すると、Alto Do、587Hz のパルス、ミッドレンジの Re、659Hz のパルスを生成することができます。ミッドレンジミー。ブザーで曲を演奏できます。
- アナログ Write () のパルス出力が固定されているため、UNO R3 ボードのアナログ Write () 機能を使用してブザーにパルスを発生させないように注意してください (500Hz)。



Connection Schematic



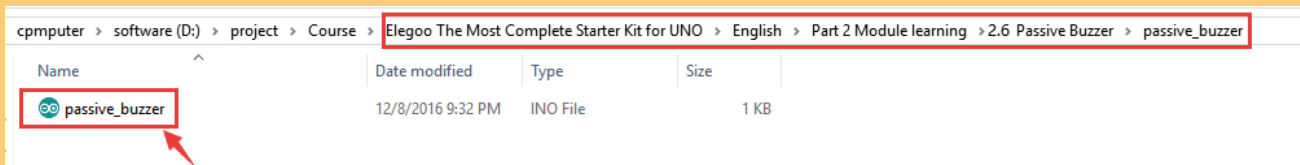
Wiring diagram



UNO R3 ボードに接続されているブザーを、赤色 (正極性) のピン 8 に黒色線 (負) を GND に配線します。

Code

■ プログラムを開いてください。



■ これを実行する前に、<pitches>ライブラリがインストールされていることを確認するか、必要に応じて再インストールしてください。そうしないと、コードは機能しません。

■ ライブラリファイルのロードの詳細については、パート1のレッスン5を参照してください。

```
int melody [] = { NOTE_C5, NOTE_D5, NOTE_E5, NOTE_F5, NOTE_G5, NOTE_A5, NOTE_B5, NOTE_C6};  
int melody [] = {...} :は配列です。
```

配列

[データ型]

説明

■ 配列は、インデックス番号でアクセスされる変数のコレクションです。C++プログラミング言語の配列Arduinoのスケッチが記述されている配列は複雑になる可能性があります、単純な配列の使用は比較的簡単です。

配列の作成(宣言)

■ 以下のすべてのメソッドは、配列を作成(宣言)するための有効な方法です。
例えば：

```
int myInts[6];  
int myPins[] = {2, 4, 8, 3, 6};  
int mySensVals[6] = {2, 4, -8, 3, 2};  
char message[6] = "hello";
```

■ myIntsのように初期化せずに配列を宣言できます。

■ myPinsでは、サイズを明示的に選択せずに配列を宣言します。

■ コンパイラは要素をカウントし、適切なサイズの配列を作成します。

最後に、mySensValsのように、配列の初期化とサイズ設定の両方を行うことができます。char型の配列を宣言する場合、必要なnull文字を保持するために、初期化よりも1つ多い要素が必要であることに注意してください。

配列へのアクセス

- 配列にはゼロのインデックスが付けられています。つまり、上記の配列の初期化を参照すると、配列の最初の要素はインデックス0にあるため、
- `mySensVals[0] == 2, mySensVals[1] == 4`, など。
- また、10個の要素を持つ配列では、インデックス9が最後の要素です。
例えば:

```
int myArray[10]={9, 3, 2, 4, 3, 2, 7, 8, 9, 11};  
// myArray[9]  contains 11  
// myArray[10]  is invalid and contains random information (other memory address)
```

- このため、配列へのアクセスには注意が必要です。(宣言された配列サイズ-1より大きいインデックス番号を使用して)配列の最後を超えてアクセスすると、他の目的で使用されているメモリから読み取られます。これらの場所からの読み取りは、無効なデータが生成されることを除いて、多くのことを行うことはおそらくありません。ランダムなメモリ位置への書き込みは間違いなく悪い考えであり、クラッシュやプログラムの誤動作などの不幸な結果につながる可能性があります。これは追跡するのが難しいバグにもなります。
- BASICやJAVAとは異なり、C++コンパイラは、配列アクセスが宣言した配列サイズの正当な範囲内にあるかどうかを確認しません。
配列に値を割り当てるには:
`mySensVals[0] = 10;`
配列から値を取得するには:
`x = mySensVals[4];`

```
tone(8, melody[thisNote], duration);
```

tone()

[高度なI/O]

説明

ピンで指定された周波数(および50%のデューティサイクル)の方形波を生成します。期間を指定できます。指定しない場合、ウェーブは`noTone()`の呼び出しまで続行されます。ピンをピエゾブザーまたは他のスピーカーに接続して、トーンを再生できます。

一度に生成できるトーンは1つだけです。別のピンでトーンがすでに再生されている場合、`tone()`を呼び出しても効果はありません。トーンが同じピンで再生されている場合、コールはその周波数を設定します。

`tone()`関数を使用すると、ピン3および11(Mega以外のボード上)のPWM出力に干渉します。

31Hz未満のトーンを生成することはできません。

Syntax

```
tone(pin, frequency)  
tone(pin, frequency, duration)
```

パラメーター

pin: トーンを生成するArduinoピン。

frequency: ヘルツ単位のトーンの周波数。使用できるデータ型: unsigned int。

duration: ミリ秒単位のトーンの持続時間(オプション)。許可されるデータ型: unsigned long。

戻り値

何もない

注意と警告

複数のピンで異なるピッチを再生したい場合は、次のピンで`tone()`を呼び出す前に、1つのピンで`noTone()`を呼び出す必要があります。